

OpenClaw 技术解剖 (三)：智能体的触手 —— 工具分类学与感知边界

mp.weixin.qq.com/s/3txjqW_OO4NTYx6ZWss4_kw

J杰克

TL;DR：

如果说 Context 是血液，Runtime 是骨架，LLM 是大脑，那么 **Tools (工具)** 就是智能体的感官与肢体。没有工具，Agent 只是一个被困在机箱里的哲学家。OpenClaw 通过一套 "原生+扩展" 的工具体系，赋予了 AI 看见世界（浏览器）、操控系统（终端）、表达自我（Canvas）以及无限进化的能力（MCP）。



1. 存在的边界：工具即感知 (Tools as Senses)

在 OpenClaw 的世界观里，Tool Definition 不仅仅是 API 列表，它是 Agent 物理存在的边界。

- 如果给了它 `web_search`，它就有了**视力**，能看到互联网的尽头。
- 如果给了它 `fs_read`，它就有了**触觉**，能感知文件系统的纹理。
- 如果给了它 `bash_exec`，它就有了**神力**，能像造物主一样操控操作系统。

Runtime 在启动时，会将这些能力的 "神经接口" (JSON Schema) 接入到 Agent 的大脑 (Context) 中。

2. 原生肢体 : Built-in Tools

OpenClaw 并不依赖仅仅一个通用的 "Run Python" 函数，而是精心设计了一套符合人类直觉的原生工具链，我们称之为 "标准生物组件"。

2.1 沟通与社会化 (The Mouth & Ears)

Agent 不是孤独的。它天生具备在多模态信道中交流的能力。

- **message / sessions_send**: 这是 Agent 的声带。它不仅能回复文本，还能发送富媒体卡片。
- **agents_list / sessions_spawn**: 这是 Agent 的社会化器官。它允许 Agent 感知到同类的存在，甚至通过 **spawn** 繁殖出子 Agent (Sub-agents) 来分担任务。这构成了 Swarm 智能的基础。
- **sessions_history**: 它是 Agent 的即时记忆检索，允许它查阅过去的对话流，保持语境的连续性。

2.2 感知与浏览 (The Eyes)

为了打破 LLM 的 "训练数据截止日期" 诅咒，OpenClaw 赋予了它实时连接世界的能力。

- **browser**: 这不是简单的 HTTP GET。这是一个全功能的 **Headless Chrome**。Agent 可以点击按钮、填写表单、滚动页面，甚至处理复杂的 SPA (单页应用) 渲染。它是 OpenClaw 上网冲浪的冲浪板。
- **web_search**: 快速的信息过滤器。当 Agent 需要广度而非深度时，它会使用这个工具来获取摘要和链接。

2.3 创造与可视化 (The Imagination)

很多时候，文字是苍白的。

- **canvas**: 这是 Agent 的白板。当 Agent 需要向用户展示复杂的 UI、图表、流程图或 React 组件时，它会调用 Canvas 工具渲染一个可视化界面。这不仅仅是图片，而是可交互的代码产物。

2.4 本体感觉 (Proprioception)

- **nodes**: 这是一个特殊的工具集，赋予 Agent 跨设备感知的能力。它可以发现网络中的其他 OpenClaw 节点（如手机、IoT 设备），并调用它们的传感器（摄像头、地理位置 **location_get**、屏幕录制 **screen_record**）。这是连接物理世界的桥梁。

3. 上帝之手 : Bash Tools

如果说上述工具是精密的仪器，那么 **bash_tools** 就是一把重锤。

OpenClaw 既然名为 "Claw"，其抓手必然强劲。通过 **exec** 和 **process** 工具，Agent 获得了宿主机的 **Shell** 访问权限。

- • **exec (原子指令)**: 一次性执行命令，如 `ls -la`, `git commit`, `npm install`。
- • **process (交互进程)**: 启动一个长连接的 REPL 进程（如 Node.js 交互式环境，或 Python shell），保持状态变量。

⚠ 力量与代价：

这里体现了 OpenClaw 最激进也最危险的设计哲学：**信任但验证 (Trust but Verify)**。我们给予 Agent 这种 "修改现实" 的底层权力，是为了让它能完成真正的 Engineering 任务（重构代码、部署服务）。但这同时也要求 Part 2 中提到的 "安全边界" (Safeguards) 必须极其坚固，防止它误删 `rm -rf /` 或执行恶意脚本。

这也是为何在 OpenClaw 爆火之后，Mac mini 卖断货的原因，人们现在只能通过为它提供一个单独的运行环境才能控制它的力量！

4. 进化的假肢：原生支持 MCP 与 Skills 标准

OpenClaw 的设计哲学是不造轮子。它的工具系统并非自创的私有格式，而是从底层架构上**完全对齐 (Native Alignment)** 了业界两大核心标准。

正如用户敏锐指出的，OpenClaw 的结构与 **Anthropic Agent Skills** 和 **MCP** 的原始定义是完全一致的。这意味着 OpenClaw 本质上就是这两个标准的 **Native Runtime (原生运行时)**。

4.1 能力定义的标准：Anthropic Agent Skills

OpenClaw 的 `workspace/skills` 目录结构，就是对 Anthropic Agent Skills^[1] 理念的直接实现。

- • **结构一致性**：OpenClaw 采用标准的 **SKILL.md** 格式。
 - • **YAML Frontmatter**：对应 Skill 的配置信息（Docker 容器配置、依赖定义、命令入口）。
 - • **Markdown Body**：对应 Skill 的 **System Prompt**。这是 Anthropic 提倡的核心理念——"技能"不仅仅是代码 (Tools)，更是"教会 Agent 如何使用这些代码"的知识 (Prompts)。
- • **无缝兼容**：这意味着为 Claude 设计的标准 Skill 包，可以直接扔进 OpenClaw 的 `skills` 目录中运行。OpenClaw 能够完美解析其中的 Prompt 指导和 Tool 定义，无需任何转换。

4.2 连接协议的标准：MCP (Model Context Protocol)

在通信层，OpenClaw 的 Runtime 直接集成了 **MCP Client**。

- • **原生直连**：OpenClaw 并非通过插件去"适配" MCP，而是其核心循环 (Agent Loop) 就"讲"的是 MCP 协议 (JSON-RPC)。
- • **通用接口**：无论是本地运行的 SQLite MCP Server，还是远程部署的 Google Drive MCP Server，对 OpenClaw 来说都是原生的"肢体"。

4.3 融合：Skill 是 MCP 的交付载体 (Delivery Vehicle)

既然有了标准的 USB 设备 (MCP Servers)，我们还需要一个"快递员"把它们送到用户手中并插上电源。**Skill** 系统就扮演了这个角色。

这就是 "Skill 成为 MCP 载体" 的底层逻辑：

1. 1. 解决"怎么装" (Distribution)：

MCP Server 本质上是一个程序（二进制文件、Docker 镜像或 Node 脚本）。它不会自己安装自己。

OpenClaw 的 Skill 通过 `metadata.install` 定义了安装方式（如 `brew install`, `npm install`）。用户只需引用一个 Skill，OpenClaw 就会自动下载并配置好对应的 MCP Server。

｜ 结论：Skill 是 MCP Server 的**安装包**。

2. 2. 解决"怎么用" (Instruction)：

MCP Server 只提供了干巴巴的 API 接口（如 `query_database`）。Agent 不需要知道这就完了，它需要知道**什么时候用**，以及**有什么注意事项**。

Skill 通过 `SKILL.md` 中的 Prompts 注入了这些"软知识"。

｜ 结论：Skill 是 MCP Server 的**说明书**。

总结：OpenClaw 利用 Skill 的包管理能力来分发 MCP Server，利用 Skill 的 Prompt 能力来教 Agent 使用 MCP Server。这种结合让"站在巨人的肩膀上"成为可能——你不仅拿到了巨人的工具，还拿到了巨人的使用心得。

下期预告：

拥有了 Memory (记忆)、Brain (大脑) 和 Hands (双手)，OpenClaw 已经是一个完整的生命体。但它生活在哪里？它如何感知时间？如何理解 "自我"？

下一篇，我们将探讨 OpenClaw 的 "**灵魂容器**" (The Soul Container) —— 也就是它的 **Runtime Environment** 与 **配置哲学**。

引用链接

[1] Anthropic Agent Skills: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>